universität wien

MULTIOMICS DATA SCIENCE - REGRESSIONEN

DATA ANALYSIS

- Grouping term for many algorithms and methods to gain insight within raw data
- regression: modeling of relations between an independent (or several) numerical valiable and a dependent numerical variable
- clustering: grouping of elements according to their features or similarities
- classification: dependent variable is nominel / categorical
- principal component analysis: transformation of high-dimensional data to apply dimension reduktion outlier detection, or other follow-up methods
- statistical tests: hypothesis-driven tests, multiple testing

REGRESSION

- Given: a set of points $(x, y)_i$ and a mathematical model $f(x \mid \Omega)$ with parameter set Ω

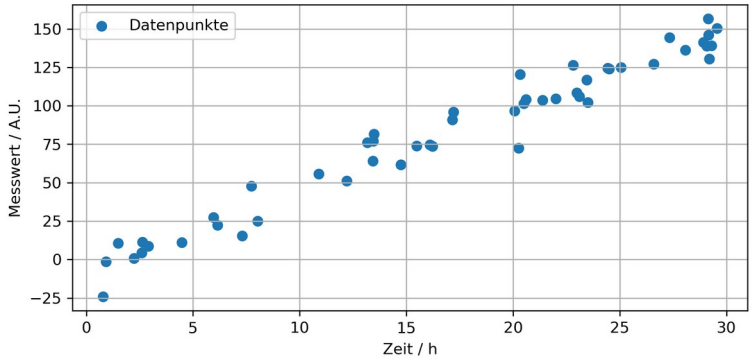
- We search for optimal parameters Ω^* so that the model describes the relation between all points $(x, y)_i$ the best way

MATHEMATICAL MODEL

- A model is a representation, that abstracts from reality with that features:
  - as precise as possible
  - with the minimal information
- A mathematical model is a model, that is based on mathematical (formal) rules, e.g. mathematical equations

LINEAR MODEL

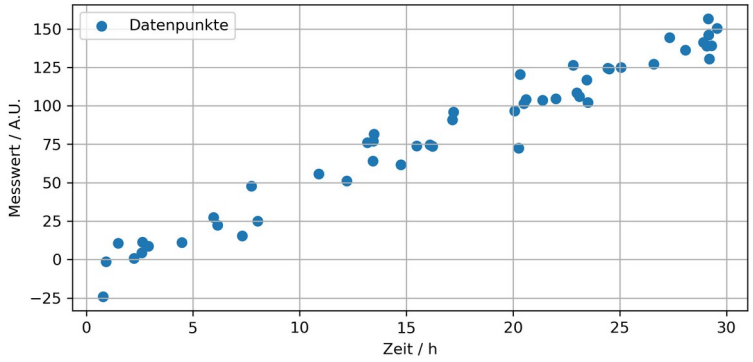
- We are given a set of points $P = (P)_i$ where each point P_i has the values (x_i, y_i)

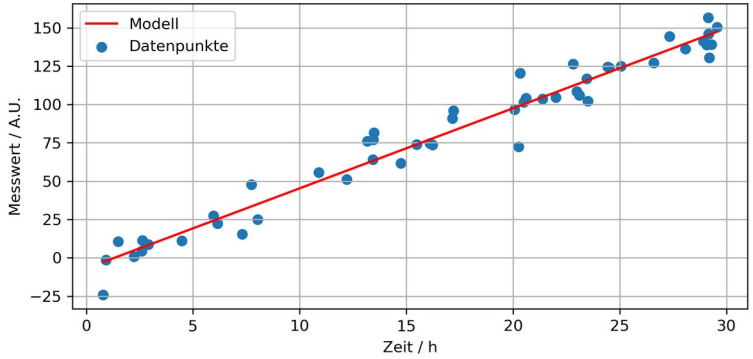


LINEAR MODEL

- We are given a set of points $P = (P)_i$ where each point P_i has the values (x_i, y_i)

- We want a linear model $f(x) = mx + b$ that describes the data points as “good“ as possible

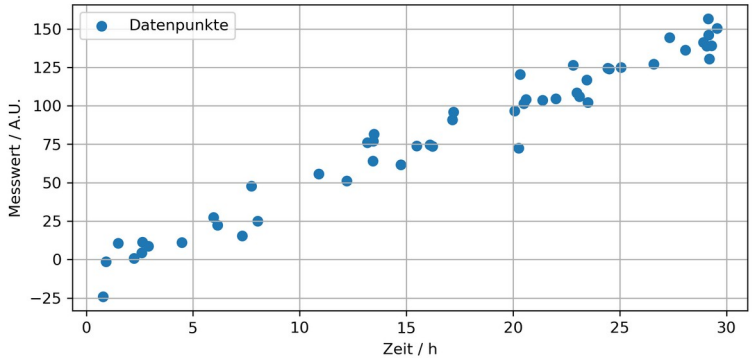


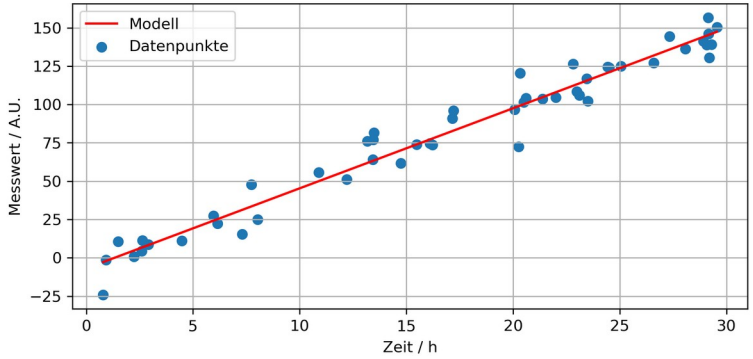


universität wien

LINEAR MODEL

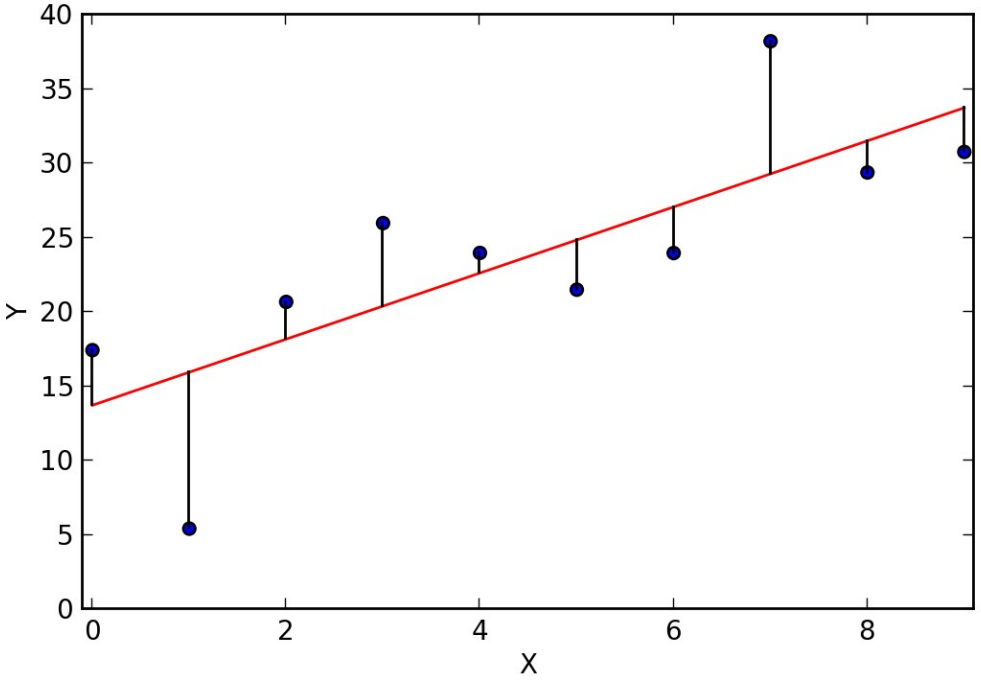
- Advantages:
  - Interpolation – what happens between the data points?
  - Extrapolation – what happens bejond the data points?
  - Data reduction – *n* data points can be described with only two parameters
  - Comparability – several data sets can be easily compared with each other





GOODNESS OF THE DESCRIPTION

- How to asses how “good“ a model describes the data?
- One way is to determine the error / difference betwenn the model to all data points



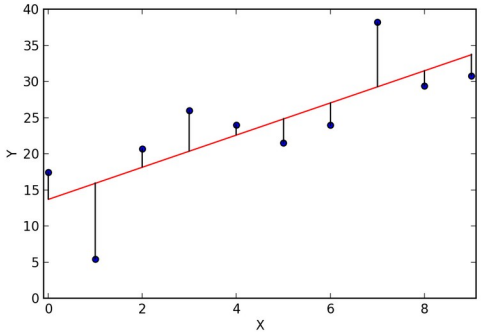
Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

universität wien

GOODNESS OF THE DESCRIPTION

- When does the model describe the data points “good“?
- When the error / difference between the model to all data points is the least
- Formal: given point $P_i = (x_i, y_i)$ and model $f(x) = mx + b$
- Error of one data point:

$$y_i - f(x_i)$$

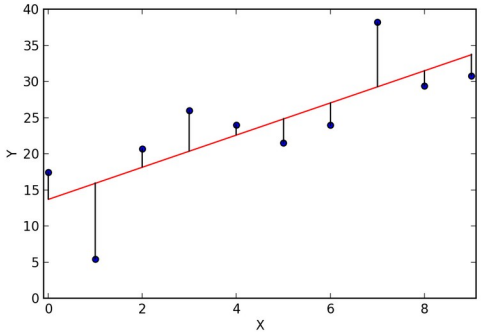


Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

GOODNESS OF THE DESCRIPTION

- When does the model describe the data points “good“?
- When the error / difference between the model to all data points is the least
- Formal: given point $P_i = (x_i, y_i)$ and model $f(x) = mx + b$
- Quadratic error of one data point:

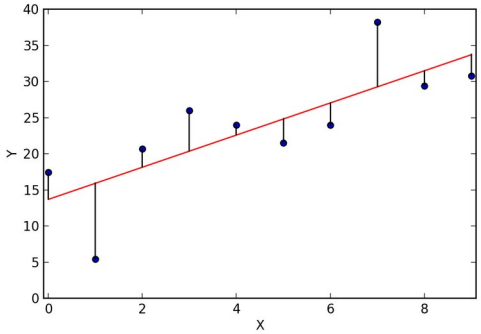
$$(y_i - f(x_i))^2$$



Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

GOODNESS OF THE DESCRIPTION

- When does the model describe the data points “good“?
- When the error / difference between the model to all data points is the least
- Formal: given point $P_i = (x_i, y_i)$ and model $f(x) = mx + b$
- Quadratic error of all data points:

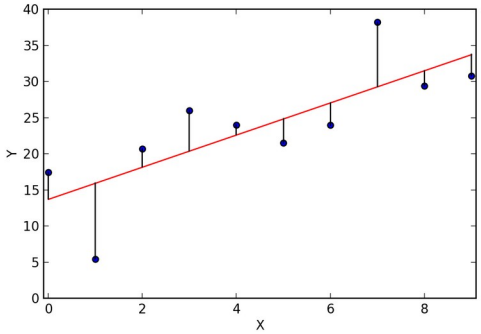


Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

$$\sum_{i}^{n} \left(y_i - f\left(x_i\right)\right)^2$$

GOODNESS OF THE DESCRIPTION

- When does the model describe the data points “good“?
- When the error / difference between the model to all data points is the least
- Formal: given point $P_i = (x_i, y_i)$ and model $f(x) = mx + b$
- Minimalizing the quadratic error of all data points:

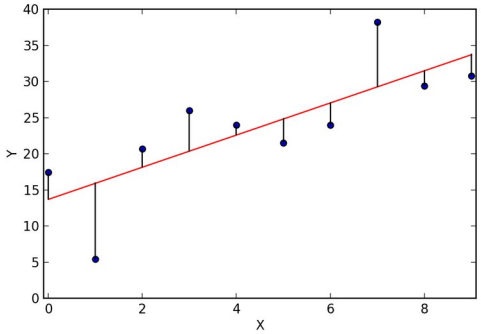


Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

$$\min \sum_{i}^{n} \left(y_i - f\left(x_i \right) \right)^2$$

GOODNESS OF THE DESCRIPTION

- When does the model describe the data points “good“?
- When the error / difference between the model to all data points is the least
- Formal: given point $P_i = (x_i, y_i)$ and model $f(x) = mx + b$
- Minimalizing the quadratic error of all data points:



Source: https://cdn-images-1.medium.com/max/1200/0*FjKhbw6Va8O8bCkF.png

$$\min \sum_{i}^{n} \left(y_i - \left(m x_i + b \right) \right)^2$$

ESTIMATING THE OPTIMAL PARAMETERS

- The parameters m, b must be estimated

$$\mathrm{d}(x, y|m, b) = \sum_{i}^{n} (y_i - (mx_i + b))^2$$

universität wien

ESTIMATING THE OPTIMAL PARAMETERS

- The parameters m, b must be estimated
- Estimation via derivation of the equation with respect to the according parameter, setting it to 0 and solving for the the parameter

$$\mathrm{d}(x, y|m, b) = \sum_{i}^{n} (y_i - (mx_i + b))^2$$

$$\frac{\partial \mathrm{d}}{\partial m} = \sum_{i}^{n} -2x_i(y_i - (mx_i + b)) = 0$$

$$\frac{\partial \mathrm{d}}{\partial b} = \sum_{i}^{n} -2(y_i - (mx_i + b)) = 0$$

universität wien

ESTIMATION OF PARAMETER B

$$\frac{\partial \mathrm{d}}{\partial b}=\sum_{i}^{n}-2\left(y_i-\left(mx_i+b\right)\right)=0$$

$$-2\sum_{i}^{n}\left(y_i-\left(mx_i+b\right)\right)=0$$

$$\sum_{i}^{n}y_i-\sum_{i}^{n}mx_i-\sum_{i}^{n}b=0$$

$$\sum_{i}^{n}y_i-m\sum_{i}^{n}x_i=nb$$

$$\frac{\sum_{i}^{n}y_i}{n}-m\frac{\sum_{i}^{n}x_i}{n}=b$$

$$\bar{y}-m\bar{x}=b$$

universität wien

ESTIMATION OF PARAMETER M

$$\frac{\partial \mathrm{d}}{\partial b}=\sum_i^n -2(y_i-(mx_i+b))=0$$

$$-2\sum_i^n (y_i-(mx_i+b))=0$$

$$\sum_i^n y_i-\sum_i^n mx_i-\sum_i^n b=0$$

$$\sum_i^n y_i-m\sum_i^n x_i=nb$$

$$\frac{\sum_i^n y_i}{n}-m\frac{\sum_i^n x_i}{n}=b$$

$$\bar{y}-m\bar{x}=b$$

$$\frac{\partial \mathrm{d}}{\partial m}=\sum_i^n -2x_i(y_i-(mx_i+b))=0$$

$$-2\sum_i^n x_i(y_i-(mx_i+b))=0$$

$$\sum_i^n x_i y_i-\sum_i^n mx_i^2-\sum_i^n bx_i=0$$

$$\sum_i^n x_i y_i-\sum_i^n mx_i^2-\sum_i^n (\bar{y}-m\bar{x})x_i=0$$

$$\sum_i^n x_i y_i-\sum_i^n mx_i^2-\sum_i^n x_i\bar{y}+\sum_i^n m\bar{x}x_i=0$$

$$\sum_i^n x_i y_i-\sum_i^n x_i\bar{y}=m\sum_i^n (x_i^2-\bar{x}x_i)$$

$$\frac{\sum_i^n x_i(y_i-\bar{y})}{\sum_i^n x_i(x_i-\bar{x})}=m$$

universität wien

LINEAR REGRESSION

- Python code for a linear regression:

$$m = \frac{\sum_i^n x_i (y_i - \bar{y})}{\sum_i^n x_i (x_i - \bar{x})} \qquad b = \bar{y} - m\bar{x}$$

```
import pandas as pd

df = pd.read_excel("measurement-points.xlsx")
x, y = df["x"], df["y"]
x_mean, y_mean = x.mean(), y.mean()

m = sum(x * (y - y_mean)) / sum(x * (x - x_mean))
b = y_mean - m * x_mean

print(m, b)
```

LINEAR REGRESSION

- Python code for a linear regression:

$$m = \frac{\sum_i^n x_i (y_i - \bar{y})}{\sum_i^n x_i (x_i - \bar{x})} \qquad b = \bar{y} - m\bar{x}$$

```
import pandas as pd

df = pd.read_excel("measurement-points.xlsx")
x, y = df["x"], df["y"]
x_mean, y_mean = x.mean(), y.mean()

m = sum(x * (y - y_mean)) / sum(x * (x - x_mean))
b = y_mean - m * x_mean

print(m, b)
```

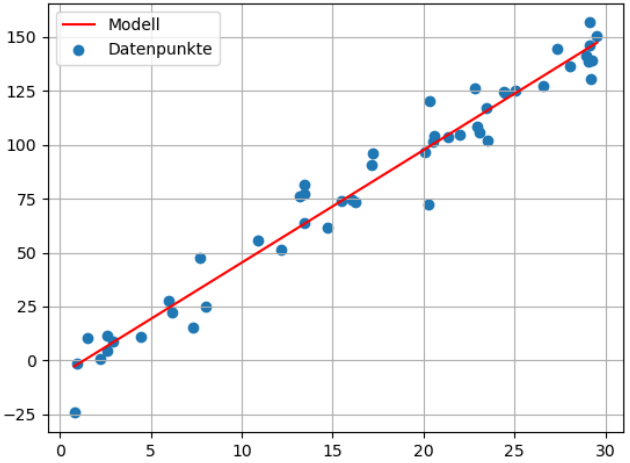
Source: https://reddit.com

LINEAR REGRESSION - PLOTED

```
import matplotlib.pyplot as plt
import numpy as np

xmod = np.arange(min(x), max(x), 0.001)
ymod = m * xmod + b

plt.scatter(x, y, label = "Datenpunkte")
plt.plot(xmod, ymod, color = "red", label = "Modell")
plt.legend()
plt.grid()
plt.show()
```

LINEAR REGRESSION - PLOTED

```
import matplotlib.pyplot as plt
import numpy as np

xmod = np.arange(min(x), max(x), 0.001)
ymod = m * xmod + b

plt.scatter(x, y, label = "Datenpunkte")
plt.plot(xmod, ymod, color = "red", label = "Modell")
plt.legend()
plt.grid()
plt.show()
```



universität wien

GOODNESS OF THE DESCRIPTION – PART 2

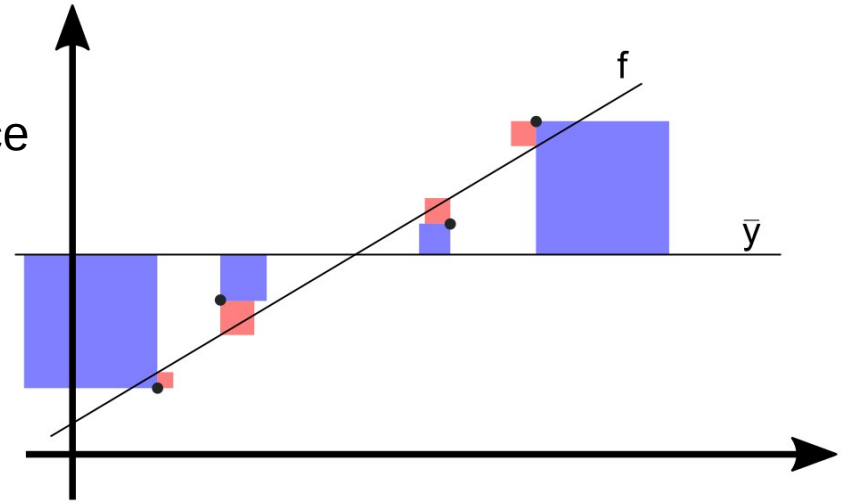
- How good describes the model the data points?
- We can compute the error between the points and the model: $\sum_{i}^{n} (y_i - (mx_i + b))^2$
- Disadvantage:
  - This value say anything and is not comparable
  - When we have more points, the value gets higher
- We need a normalization!

COEFFICIENT OF DETERMINATION R²

- For linear regressions, we can calculate the coefficient of determination

- The coefficient R² is the relation between the variance of the residuals and the variance of the data points

$$R^2 = 1 - \frac{\mathrm{Var}(f-y)}{\mathrm{Var}(y)} = 1 - \frac{\sum_i^n (f(x_i) - y)^2}{\sum_i^n (y_i - \bar{y})^2}$$



- The measure ranges between 0 and 1 (full agreement)

- Measure only applicable for **linear** functions f and an optimal parameter set

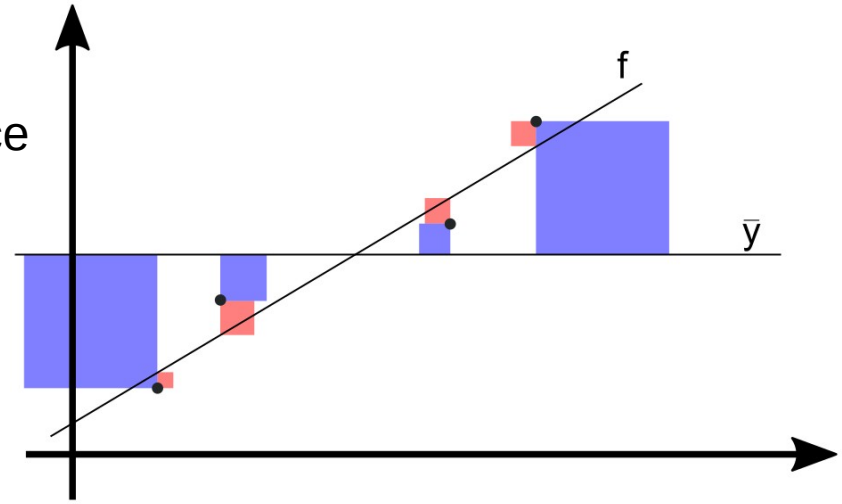
Source: https://de.wikipedia.org/wiki/Bestimmtheitsmaß

COEFFICIENT OF DETERMINATION R²

- For linear regressions, we can calculate the coefficient of determination

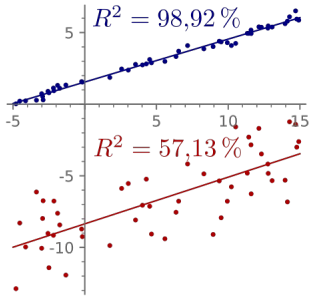
- The coefficient R² is the relation between the variance of the residuals and the variance of the data points

$$R^2 = 1 - \frac{\mathrm{Var}(f-y)}{\mathrm{Var}(y)} = 1 - \frac{\sum_i^n (f(x_i) - y)^2}{\sum_i^n (y_i - \bar{y})^2}$$



- The measure ranges between 0 and 1 (full agreement)

- Measure only applicable for **linear** functions f and an optimal parameter set



Source: https://de.wikipedia.org/wiki/Bestimmtheitsmaß

universität wien

POLYNOMIAL REGRESSION

- Polynomials of degree *n* are mathematical functions of the following form:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_2 x^2 + a_1 x + a_0$$

- A special case here is, when *n* = 1 (simple linear regression)

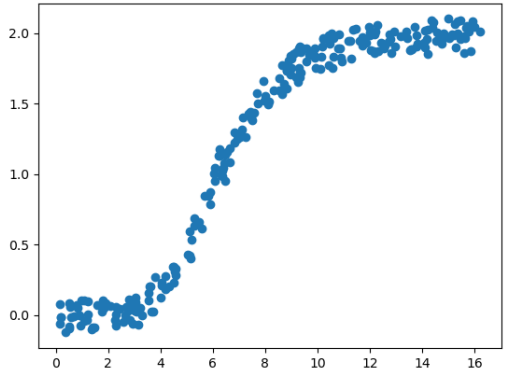
- NumPy offers here a quick method for polynomal fitting with `polyfit`:

```
import numpy as np
X = np.random.normal(5, 3, 100)
Y = np.random.normal(7, 5, 100)
degree = 2                                  # quadratic polynomial
coeff = np.polyfit(X, Y, degree)
# f(x) = coeff[0] * x**2 + coeff[1] * x + coeff[2]
print(coeff)
```

NON-LINEAR REGRESSION

- A non-linear regression is a regression, where the function parameters have a non-linear relation
- Often, such a regression can only be solved numerically, that is a step-by-step improvement, where the parameter are optimized incrementally
- Usually, this numerical approach finds a good parameter set but not the optimal
- Again, Python offers an “easy“ method for non-linear regression

NON-LINEAR REGRESSION



- Hill curves are sigmoidic functionen with following function:

$$h(x)=\frac{s\cdot x^f}{k^f+x^f}$$

  - s = maximal elongation
  - k = position of the half clongation
  - f = Hill coefficient

- The parameter *f* is here an exponent and no linear coefficient, such as parameter *s*

- Hill curves are often used to model processes in biochemistry and pharmacology

- We will perform a non-linear regression with the library SciPy

universität wien

NON-LINEAR REGRESSION

$$h(x)=\frac{s\cdot x^f}{k^f+x^f}$$

- SciPy offers a function `leastsq` for the follwing optimization:

$$\hat{p}=\arg\min\sum(f(p,x,y))^2$$

universität wien

NON-LINEAR REGRESSION

$$h(x)=\frac{s\cdot x^f}{k^f+x^f}$$

- SciPy offers a function `leastsq` for the follwing optimization:

$$\hat{p}=\arg\min\sum(f(p,x,y))^2$$

- We need first to define a residual function, that has to be minimized:

```
def hill_res_func(par, X, Y):                # X, Y are numpy arrays
    s, k, f = par
    return Y - s * X**f / (k**f + X**f)
```

NON-LINEAR REGRESSION

$$h(x)=\frac{s\cdot x^f}{k^f+x^f}$$

- SciPy offers a function `leastsq` for the follwing optimization:

$$\hat{p}=\arg\min\sum(f(p,x,y))^2$$

- We need first to define a residual function, that has to be minimized:

```
def hill_res_func(par, X, Y):                # X, Y are numpy arrays
    s, k, f = par                            # the function leastsq does the
    return Y - s * X**f / (k**f + X**f)      # squaring and summation
```

universität wien

NON-LINEAR REGRESSION

$$h(x)=\frac{s\cdot x^f}{k^f+x^f}$$

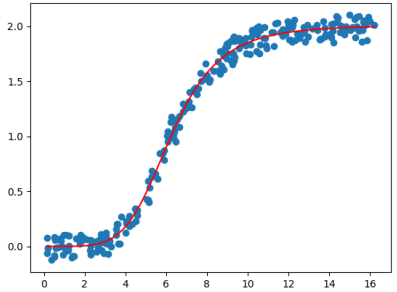
- SciPy offers a function `leastsq` for the follwing optimization:

$$\hat{p}=\arg\min\sum(f(p,x,y))^2$$

- We need first to define a residual function, that has to be minimized:

```
def hill_res_func(par, X, Y):                # X, Y are numpy arrays
    s, k, f = par                            # the function leastsq does the
    return Y - s * X**f / (k**f + X**f)      # squaring and summation

from scipy.optimize import leastsq
start_par = [1, 1, 1]                # pick ‘reasonable‘ start parameters
result = leastsq(hill_res_func, start_par, args = (X, Y))
s_opt, k_opt, f_opt = result[0]   # optimal parameter set
```

NON-LINEAR REGRESSION



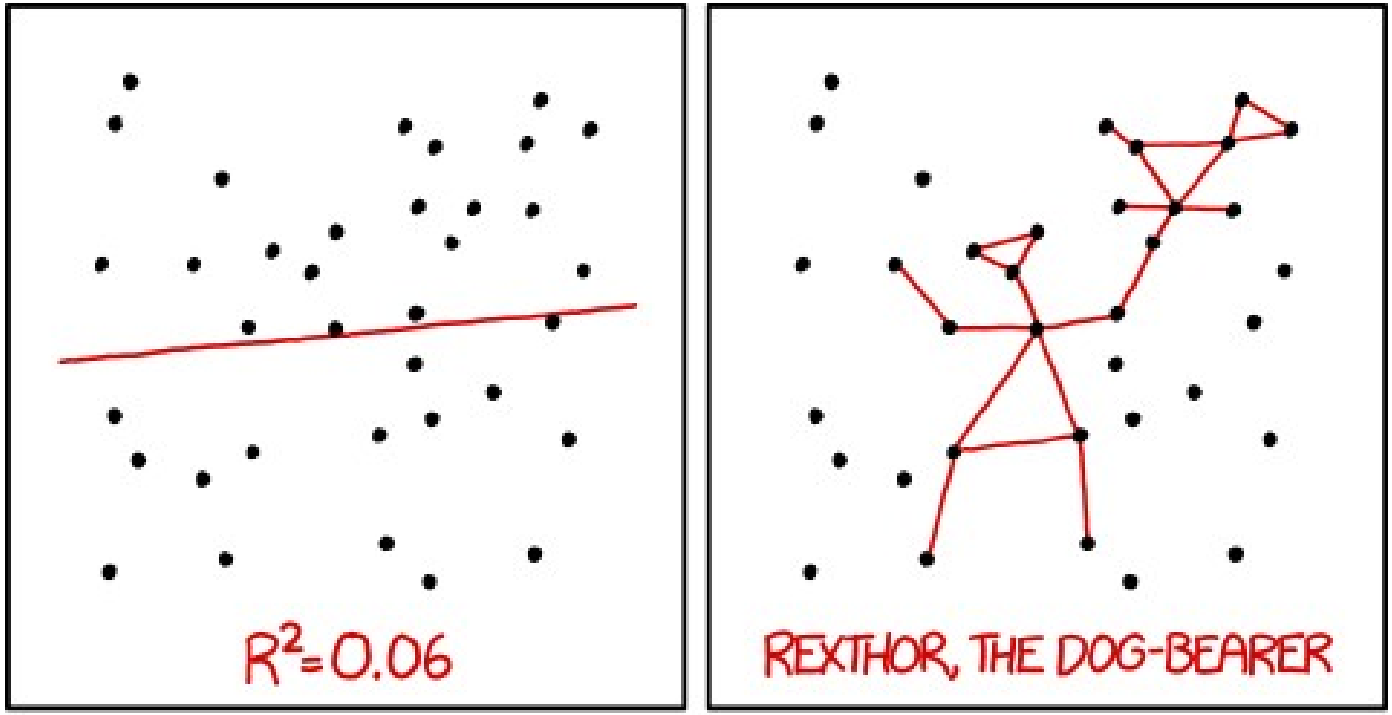
- SciPy offers a function `leastsq` for the follwing optimization:

$$\hat{p} = \arg\min \sum (f(p, x, y))^2$$

- We need first to define a residual function, that has to be minimized:

```
def hill_res_func(par, X, Y):                # X, Y are numpy arrays
    s, k, f = par                            # the function leastsq does the
    return Y - s * X**f / (k**f + X**f)      # squaring and summation

from scipy.optimize import leastsq
start_par = [1, 1, 1]                # pick ‘reasonable‘ start parameters
result = leastsq(hill_res_func, start_par, args = (X, Y))
s_opt, k_opt, f_opt = result[0]   # optimal parameter set
```

universität wien



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER TO GUESS THE DIRECTION OF THE CORRELATION FROM THE SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.